命令式编程 vs 声明式编程

核心区别

命令式编程（Imperative）：告诉计算机**“怎么做”**（How）—— 一步步描述操作步骤和状态变化。

声明式编程（Declarative）：告诉计算机**“要什么”**（What）—— 描述目标结果，具体实现交给底层。

用一个类比：

- 命令式：告诉厨师“先切菜、再热油、下锅翻炒3分钟、加盐出锅”
- 声明式：告诉厨师“我要一盘宫保鸡丁”

对比表格

维度	命令式	声明式
关注点	How（过程/步骤）	What（结果/目标）
状态管理	显式维护状态变量	通常隐藏状态
代码风格	循环、赋值、条件分支	表达式、组合、高阶函数
可读性	步骤清晰但冗长	简洁但需理解抽象
副作用	大量显式副作用	通常尽量无副作用
调试	可逐步跟踪	较难跟踪内部执行

代码示例对比

任务：从一个列表中筛选出偶数并乘以 2

命令式（Python）

```
nums = [1, 2, 3, 4, 5, 6]
result = []
for n in nums:
    if n % 2 == 0:
        result.append(n * 2)
print(result)  # [4, 8, 12]
```

> 明确写出了：创建空列表 → 遍历 → 判断 → 追加。**每一步怎么做都写清楚了。**

声明式（Python）

```
nums = [1, 2, 3, 4, 5, 6]
result = [n * 2 for n in nums if n % 2 == 0]
print(result)  # [4, 8, 12]
```

> 只描述"要偶数乘2的结果"，循环细节被隐藏。

声明式（SQL）—— 最典型的声明式语言

```
SELECT id, name FROM users WHERE age > 18 ORDER BY name;
```

> 你只说"要什么数据"，数据库引擎决定怎么扫描、怎么排序、用不用索引。

对应的编程语言例子

偏命令式

- **C**：典型的过程式/命令式，指针、手动内存管理、逐句执行

- **Pascal / Fortran**：过程式
- **汇编语言**：最纯粹的命令式
- **Java / C#**（传统写法）：面向对象 + 命令式语句为主
- **Go**：显式循环、显式错误处理，风格偏命令式

偏声明式

- **SQL**：查询语言，描述结果集
- **HTML / CSS**：描述"页面长什么样"，而非"怎么绘制"
- **Prolog**：逻辑编程，声明事实和规则
- **Haskell**：纯函数式，无副作用
- **正则表达式**：描述匹配模式，而非匹配算法

混合/多范式（现代语言常见）

- **Python / JavaScript / Ruby**：既支持 `for` 循环（命令式），也支持 `map/filter/reduce`、列表推导（声明式）
- **Scala / Kotlin**：面向对象 + 函数式
- **Rust**：命令式为主，但迭代器链是声明式风格

同一个 JS 例子：

```
// 命令式
let result = [];
for (let i = 0; i < nums.length; i++) {
  if (nums[i] % 2 === 0) result.push(nums[i] * 2);
}

// 声明式
const result = nums.filter(n => n % 2 === 0).map(n => n * 2);
```

什么时候用哪种？

- **命令式**适合：需要精细控制性能、底层操作、复杂状态机、算法实现（如手写排序）。
- **声明式**适合：数据查询/转换、UI 描述、配置、并行/分布式（如 Spark）、业务规则表达。
- 实践中**大多数项目是混合的**：用声明式表达"意图"，用命令式处理"性能关键路径"。

> 一句话总结：**命令式描述过程，声明式描述结果。**声明式通常建立在命令式之上（比如 SQL 引擎内部仍是命令式实现），是更高层的抽象。